

Subprograms: Generic Types

Programming Languages

Millersville University

Outline

- OCaml / F#
 - Type Variables
- Java
 - Generics
 - Wildcards
- C++
 - Templates
 - SFINAE
 - Concepts
- Scripting Languages

Generic Subprograms

- A *generic (or polymorphic) subprogram* takes parameters of different types on different activations
 - `sort (int[] A)` vs `sort (double[] A)`
- Overloaded subprograms provide *ad hoc polymorphism*
 - `square (int x)` vs `square (double x)`
 - Overloads can behave differently
- A subprogram that takes a generic type parameter provides *parametric polymorphism*
 - `sort (T[] A)`

OCaml / F# Type Variables

```
type 'a option = Some of 'a | None  
type int_option = int option
```

```
compare : 'a -> 'a -> int  
(parametric function)
```

- 'a is a *type variable*, and option is a (*parametric*) *type constructor*
- All type variables start with a single quote
- There's a bunch more to OCaml type theory that we are NOT going to cover

Java Generics

- *Type Parameters* allow us to specify “any type” in Java
 `ArrayList<Number> lst = ...`
- Converted to code that operates on *Object-s*

```
public class Node<E> {  
    private E      data;  
    private Node<E> next;  
}
```

Java Generics with Constraints

- Type parameters can be constrained to be subtypes of another type
- Called an upper bound
- Decays to the constraint type (Comparable<E>)

```
class BstNode<E extends Comparable<E>> {  
    private E          data;  
    private BstNode<E> left;  
    private BstNode<E> right;  
}
```

Java Wildcards

- *Wildcards* are similar to type parameters, but also support lower bounds
- ? decays to Object type; <T> → you name and use the type
- ? : you don't care about the type; <T>: you name and use the type

```
public static int size(Collection<?> obj) {  
    return obj.size();  
}
```

Java Wildcards with Constraints

- We can specify a type upper bound with wildcards
 - `extends X`
 - decays to X; some unknown subtype of X
- Can also specify a type lower bound with wildcards
 - `super Y`
 - decays to Object; some unknown supertype of Y

```
public void foo(List<? extends Number> nums)
```

```
public void bar(List<? super Integer> list)
```

Java Generics + Wildcards

- Wildcards or type parameters can impose an **upper bound**
- Wildcards can impose a **lower bound**
- Full type safety!

```
public static <T extends Number>
void
adder(T elem, List<? super Number> list)
{
    list.add(elem);
}
```

C++ Templates

- *Templates* support generic programming
 - Instantiated once for each argument combination, *unlike* Java generics; e.g., `add<int>` and `add<double>`, C++ compiler generates two functions (two real implementations in machine code)
- Classes and functions can have template parameters
- Template Parameters can be any of:
 - Type (including enumeration type)
 - Enumerator
 - Integral Constant
 - Floating-point Constant
- Template parameters are specified with the `template` keyword at definition, and contained within `<>` at the call site/usage

C++ Class Templates

```
template<typename T>
class
vector {
    T*   data;
    int  size;
    int  capacity;
public:
    . . .
};
```

Whatever type T has
is the **exact type** that
is used throughout

C++ Class Templates:

Default Type Params

```
template<
    typename T,
    typename Alloc = std::allocator<T>>
class
vector {
    T*   data;
    int  size;
    int  capacity;
public:
    . . .
};
```

If we omit the second type parameter, `std::allocator<T>` will be used for `Alloc`

C++ Function Templates

```
template<typename T>
T
max (T a, T b) {
    if (a > b)
        return a;
    return b;
}
```

Both parameters are
passed by value!

BTW, `std::vector`
is comparable

C++ Function Templates

```
template<typename T>
```

```
T
```

```
max (T const& a, T const& b) {
```

```
    if (a > b)
```

```
        return a;
```

```
    return b;
```

```
}
```

The result is still
copied back!

C++ Function Templates

```
template<typename T>
T const&
max (T const& a, T const& b) {
    if (a > b)
        return a;
    return b;
}
```

No extra copies
made now

C++ Templates: Type Constraints

```
template<typename T>
```

```
class
```

```
vector {
```

```
public:
```

```
    template<typename Iter>
```

```
    vector (Iter first, Iter last);
```

```
    vector (size_t size, T const& value);
```

```
    ...
```

```
};
```

What if `T == size_t` ?

C++ Templates: Type Constraints

```
template<typename T>
```

```
class
```

```
vector {
```

```
public:
```

```
    template<typename Iter>
```

```
    vector (Iter first, Iter last);
```

```
    vector (size_t size, size_t const& value);
```

```
    ...
```

```
};
```

What if `T == size_t` ?



Template overload resolution failure!
Multiple candidates found

C++ Templates: Type Constraints

`template<typename T>` where `T = size_t`

`class`

`vector {`

`public:`

`template<typename Iter>`

`vector (Iter first, Iter last);`

`vector (size_t size, size_t const& value);`

`...`

`};`

Solution:

Constrain the range-constructor to only work with iterators



C++ Templates: Type Constraints

- An Iterator is Dereferencable and Incrementable

Type Constraint:

```
decltype(  
    *std::declval<Iter>(),      // *iter  
    ++std::declval<Iter>()     // ++iter  
)
```

- `decltype(e)`: the evaluated type of `e`
- `std::declval<T>`: give me an instance of `T`

If it fails, then it's not going to yield a compiler error

C++ Templates: Type Constraints

```
template<typename T>
class
vector {
public:
    template<
        typename Iter,
        typename = decltype(
            *std::declval<Iter>(),
            ++std::declval<Iter>())>
    vector (Iter first, Iter last);

    vector (size_t size, T const& value);
    ...
};
```

SFINAE:
Substitution
Failure
Is
Not
An
Error

So if `Iter = size_t`:

`*size_t` is invalid.
`++size_t` may be valid for a variable, but dereference is not.

The range constructor is rejected.

That leaves only the fill constructor.

C++ Templates: Concepts

- New for C++20

```
template<typename T>
class
vector {
public:
    template<std::input_iterator Iter>
    vector (Iter first, Iter last);

    vector (size_t size, T const& value);
    ...
};
```

Scripting Languages

- Scripting languages are arguably the most generic
- Why?
 - No type definitions are required
 - Often interpreted, so unreachable code is never evaluated

Attribution

Slides originally developed by William Killian,
modified by Stephanie Schwartz

modified by Gary Zoppetti

modified by Jingnan Xie

Sources include Robert Sebesta's [Concepts of Programming Languages](#)